Facultad de Economía,
Empresa y Turismo
Universidad de La Laguna

Haunted Garden

Motores y Sistemas de Desarrollo para Videojuegos

Fabián González Lence - alu0101549491@ull.edu.es

Table of Contents

1. Introduction **2**
2. Map Obstacles (2 points) **2**
3. Jumping Mechanic (5 points) **8**
4. Change of Materials (2 points) **12**
4.1. Ground 12
4.2. Ceiling 13
4.3. Walls 14
5. Treasure Collectables (3 points) **16**
6. Final Result **20**
7. Bibliography **21**

1. Introduction

Haunted Garden is an Unreal Engine game developed as part of the course. It features core gameplay mechanics where the player collects points from mushrooms, while strategically evading pursuing ghosts that attempt to drain the player's soul indicator bar.

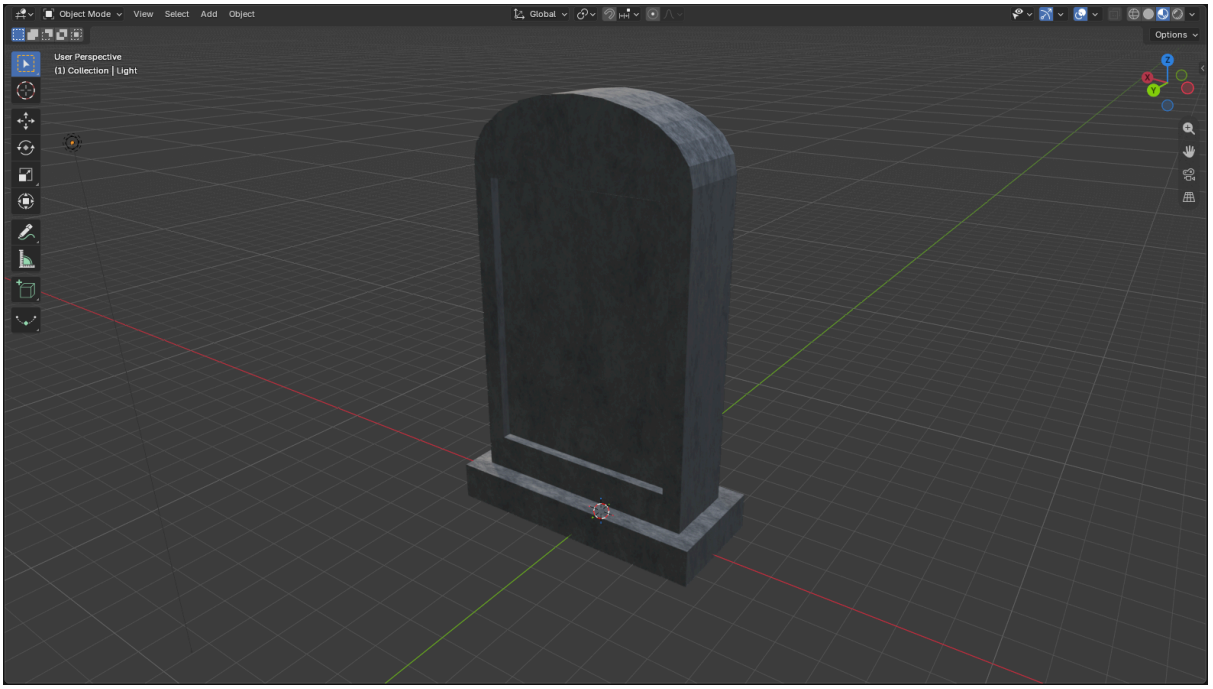
In this project report, we'll see a couple upgrades that have been made to the original project in order to create a more original proposal, with complex mechanics based on the existing game flow.

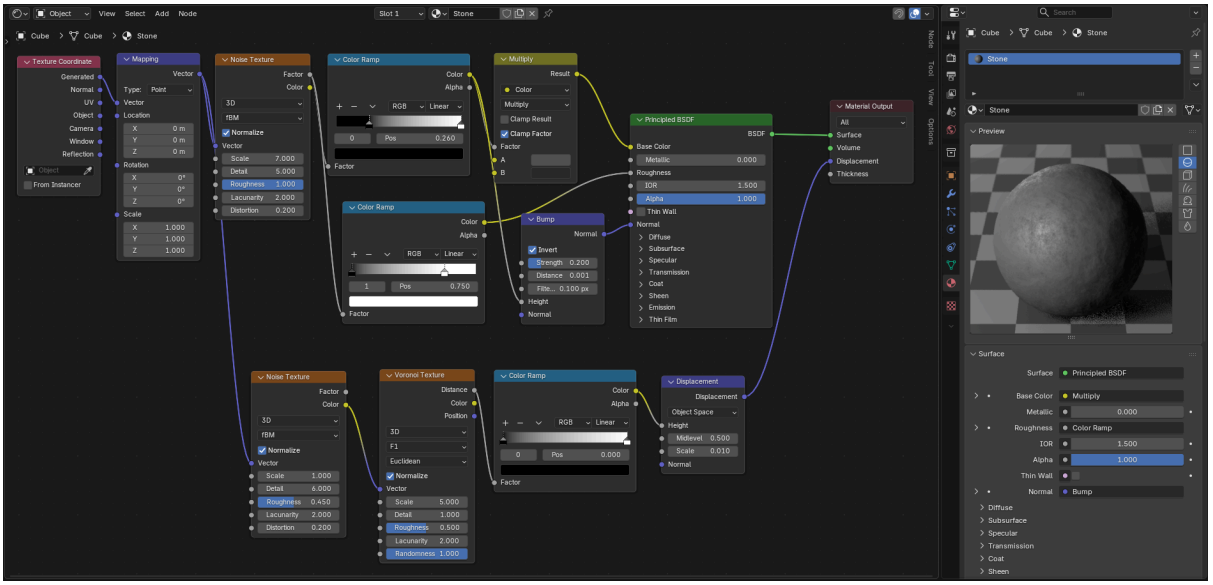
2. Map Obstacles (2 points)

The first thing that comes to mind is that the map looks pretty empty, it only has three elements: The player, the ghosts and the mushrooms. So, to solve this, we'll introduce a couple of obstacles that make the movement of all the characters more difficult in the scene.

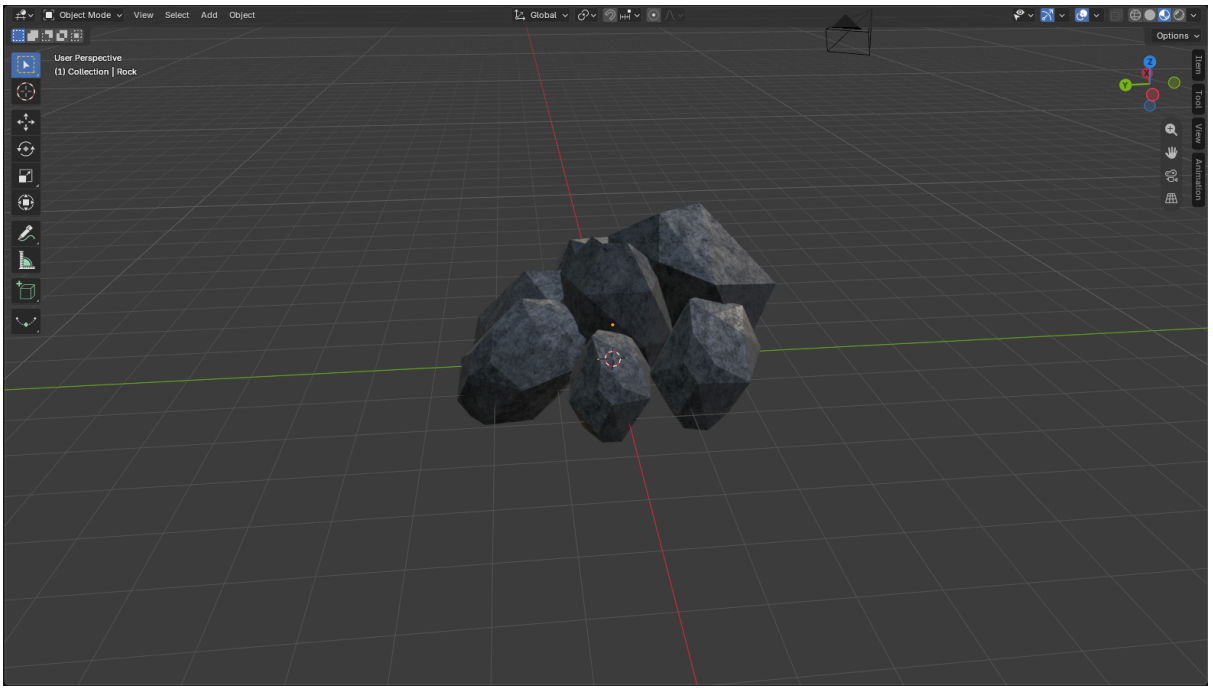
For that, we'll need to create a couple simple assets in Blender that will be imported into the Unreal Engine project: **A Tombstone, a Rock Pile and a Pumpkin.**

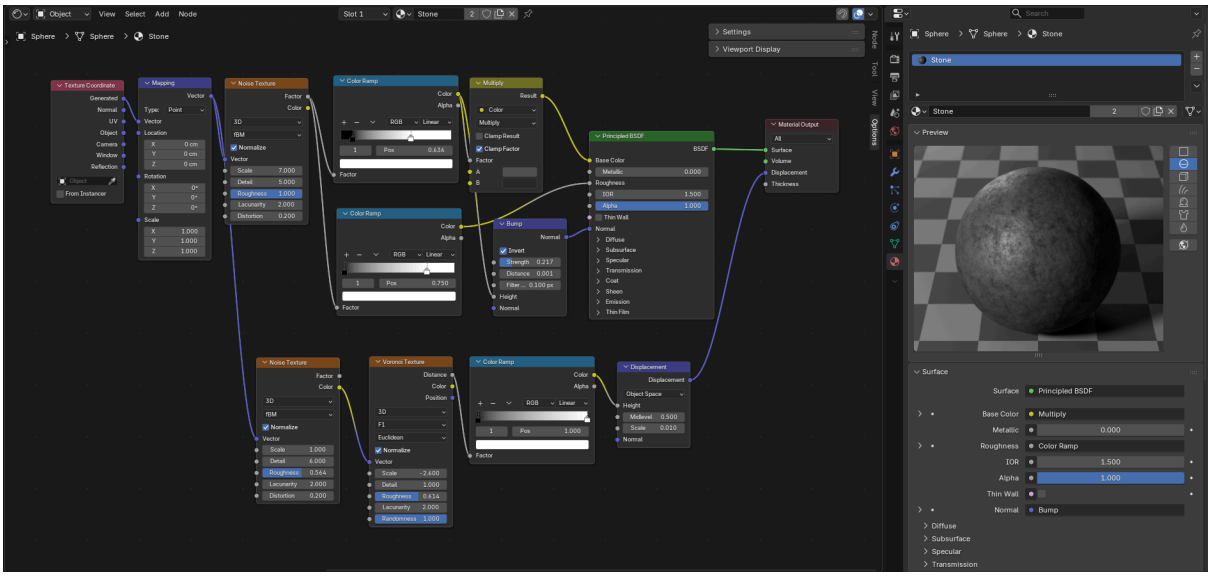
For the Tombstone, based on a cube, we used a couple of modifiers like Mirror or Boolean to create a simple tombstone asset and a custom material that would be used as inspiration for the material of the Rock Pile, based on this video [1].



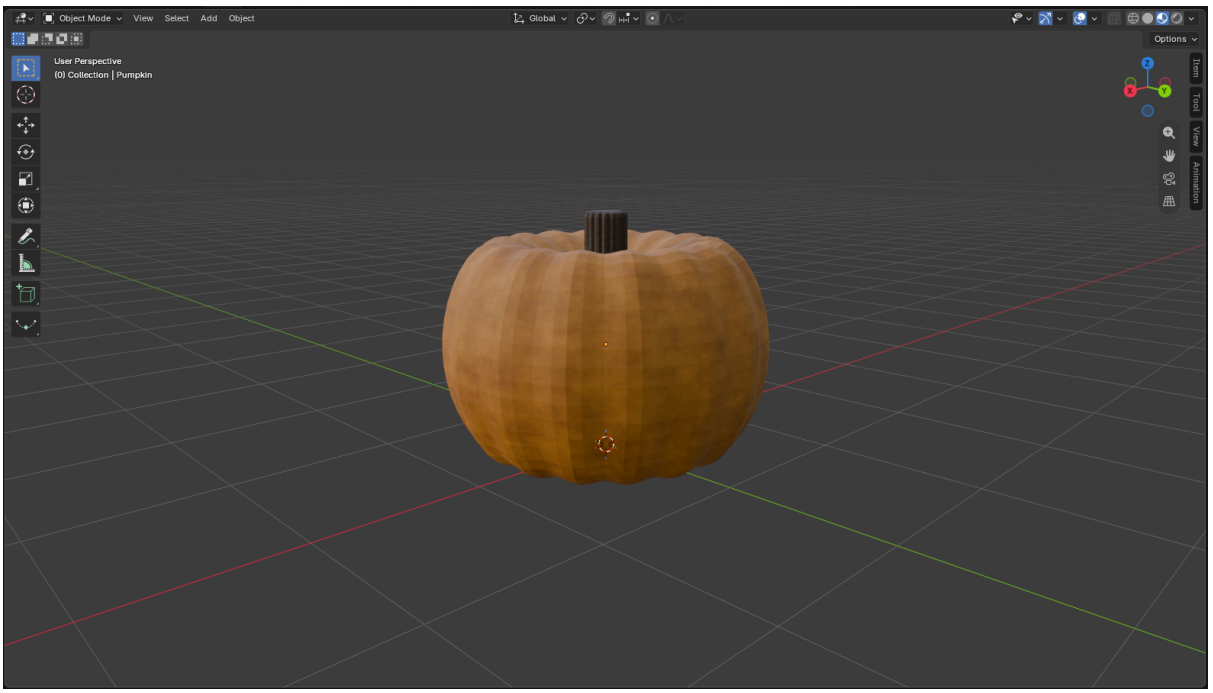


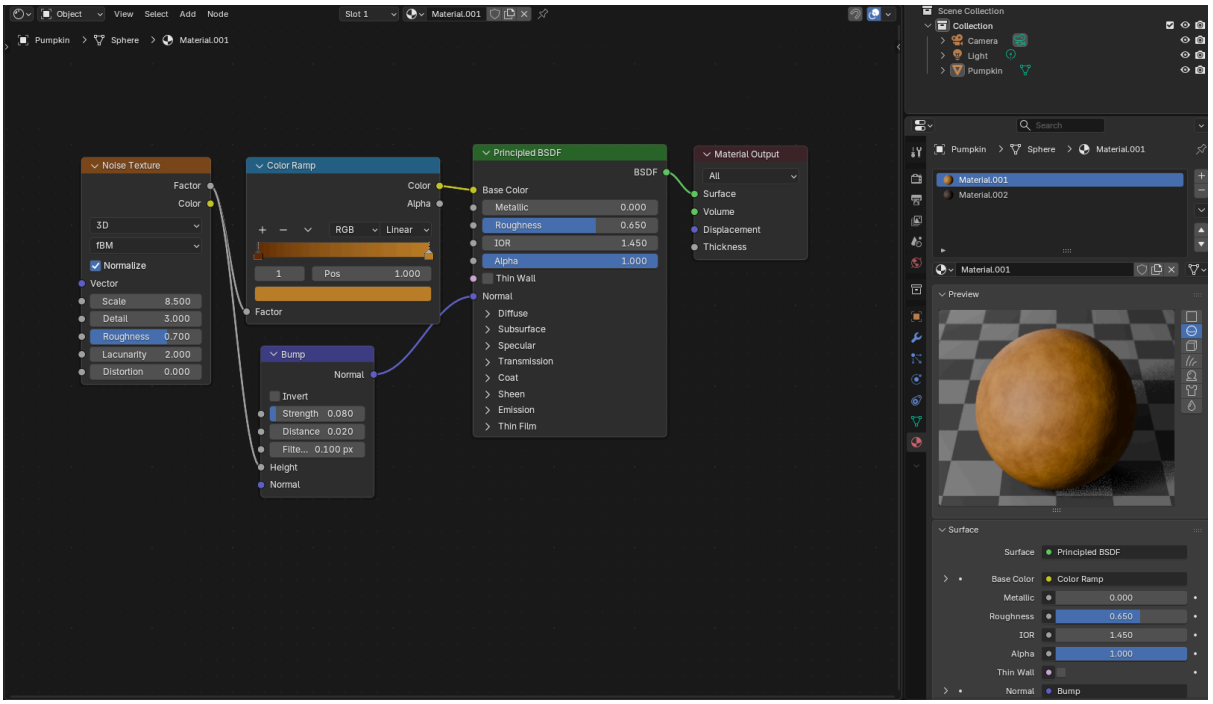
For the Rock Pile, by modifying a Ico Sphere, cutting its edges, extruding faces, moving vertices and duplicating the base rock, we achieved a low polygon Rock Pile with a similar material as the Tombstone.

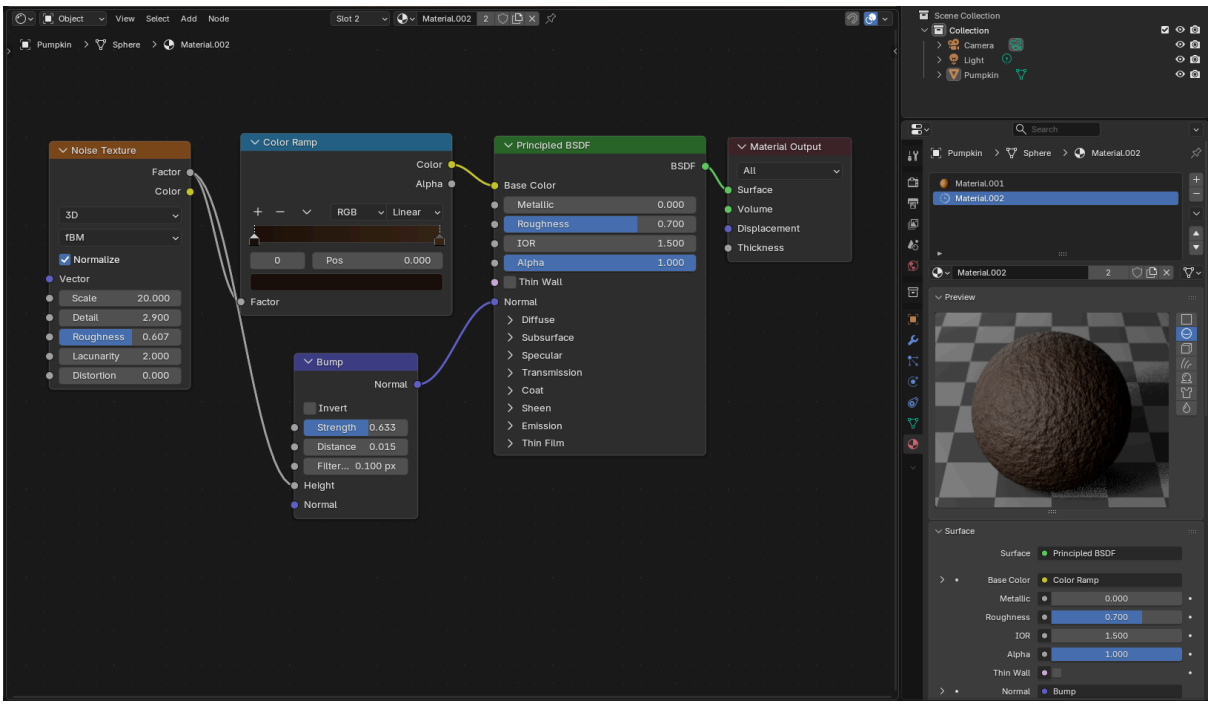




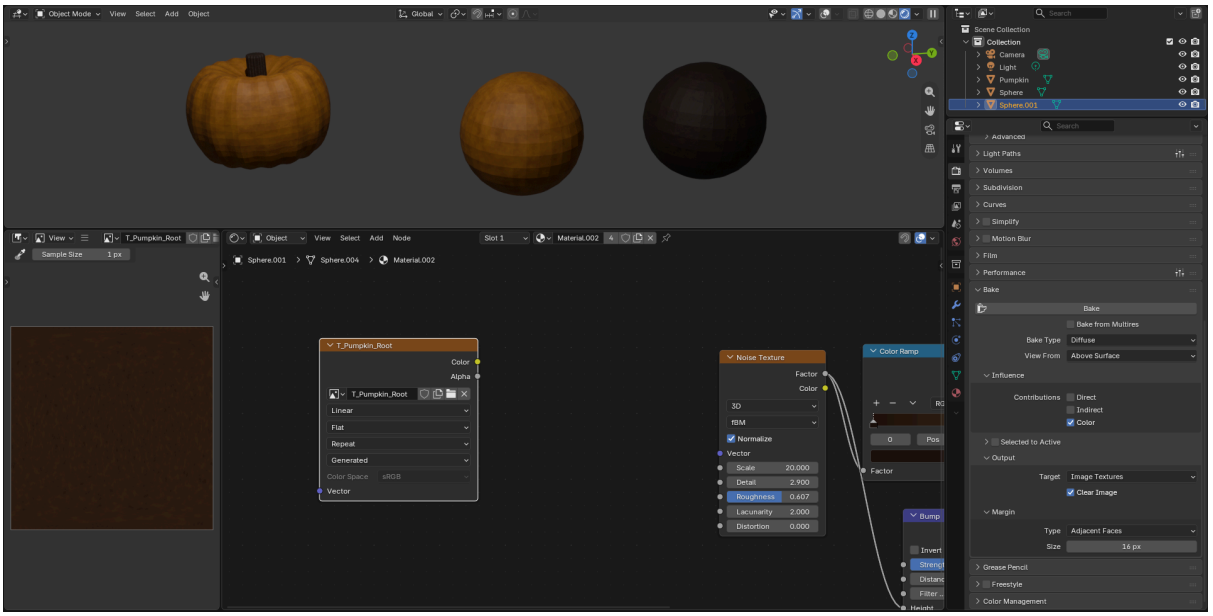
Finally, for the Pumpkin, based on the operations used on this video [2], we created a simple object that resembles a Pumpkin. The materials used for this asset, simply uses a basic scale of colors of orange and brown, also uses noise textures to make it more visually appealing.

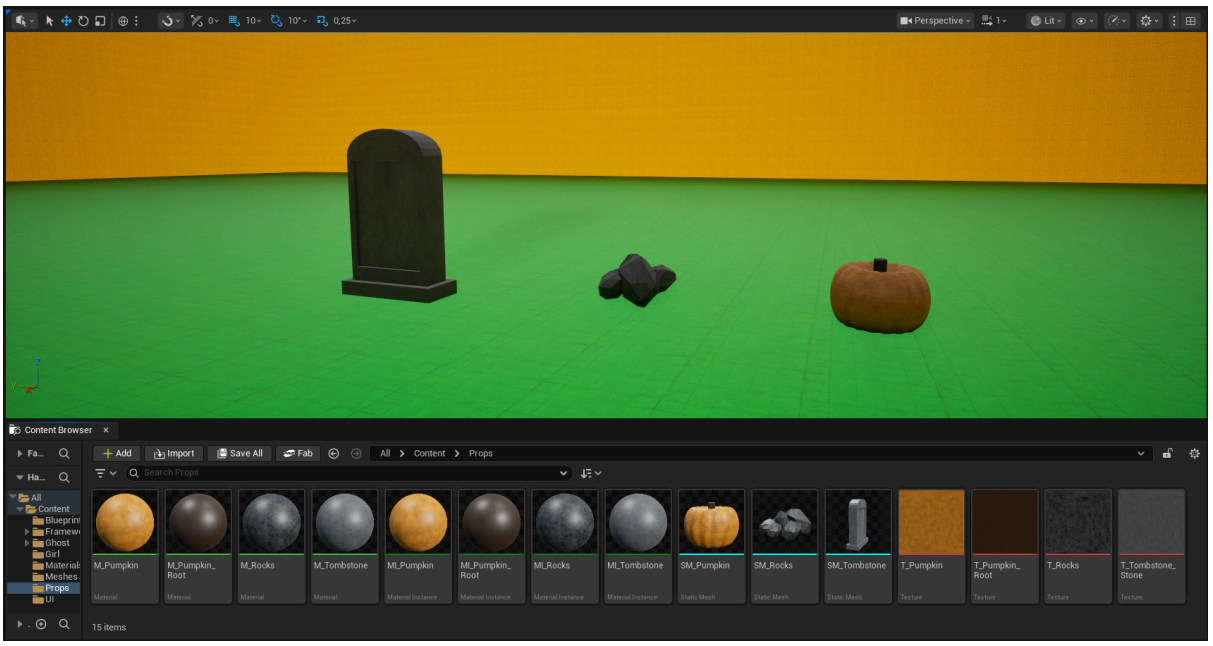

Object Mode
View
Select
Add
Object
Global
Options
User Perspective
(0) Collection | Pumpkin
Item
Tool
View
Animation


Object
View
Select
Add
Node
Slot 1
Material.001
Pumpkin
Sphere
Material.001
Noise Texture
Factor
Color
3D
fBM
Normalize
Vector
Scale 8.500
Detail 3.000
Roughness 0.700
Lacunarity 2.000
Distortion 0.000
Color Ramp
Color
Alpha
RGB
Linear
1
Pos 1.000
Factor
Principled BSDF
BSDF
Base Color
Metallic 0.000
Roughness 0.650
IOR 1.450
Alpha 1.000
Thin Wall
Normal
Diffuse
Subsurface
Specular
Transmission
Coat
Sheen
Emission
Thin Film
Material Output
All
Surface
Volume
Displacement
Thickness
Bump
Normal
Invert
Strength 0.080
Distance 0.020
Filte... 0.100 px
Height
Normal
Scene Collection
Collection
Camera
Light
Pumpkin
Search
Pumpkin
Sphere
Material.001
Material.001
Material.002
Material.001
Preview
Surface
Surface
Principled BSDF
Base Color
Color Ramp
Metallic 0.000
Roughness 0.650
IOR 1.450
Alpha 1.000
Thin Wall
Normal
Bump

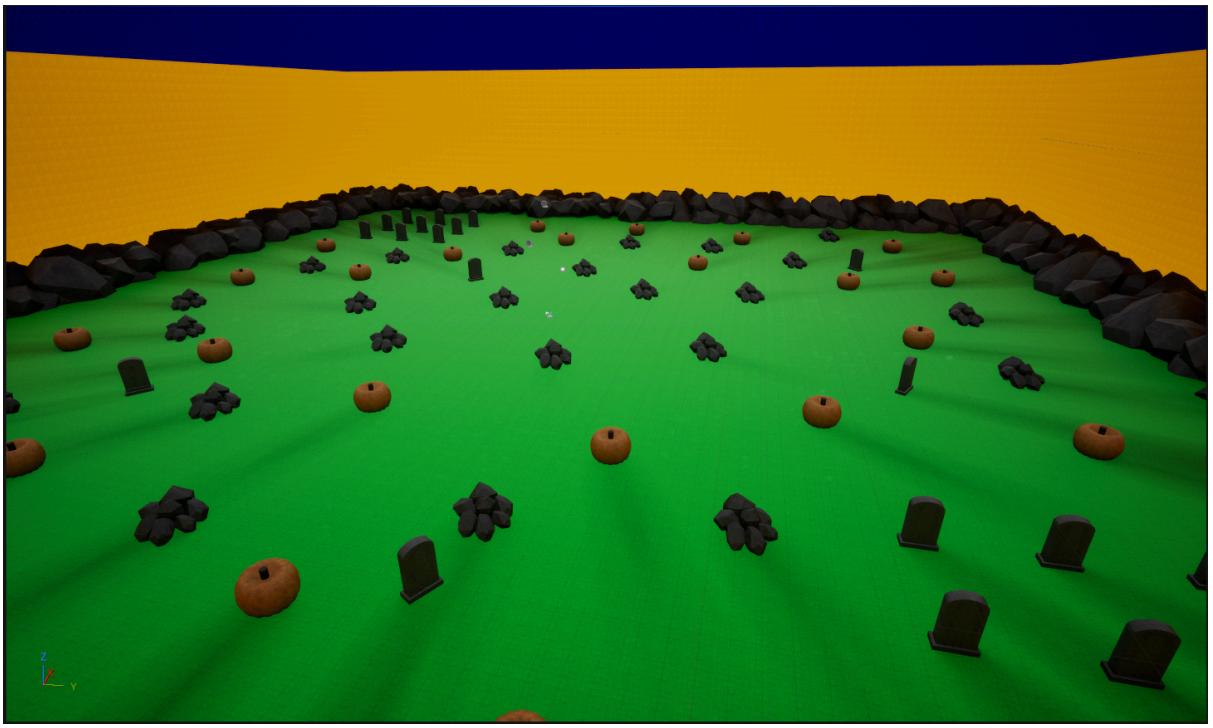


Once we had all three assets we imported them to Unreal Engine, in order to place a couple of them in the original map. The process of importing objects from Blender into Unreal Engine posed a problem because the materials weren't exported correctly along with the FBX file, so we had to bake the textures created in Blender as PNG files in order to import them correctly and apply them to a custom material.





Unreal Engine made a predefined collider for each asset, making possible to use a simplified collision to calculate the collisions between the character and the props in the scene. After placing a couple of props of the new assets introduced and creating a wall of rocks that covers the border of the garden, the map of Haunted Garden has a more complex look and delivers a higher difficulty level to the player, who has to maneuver between the obstacles in order to escape from the ghosts.



3. Jumping Mechanic (5 points)

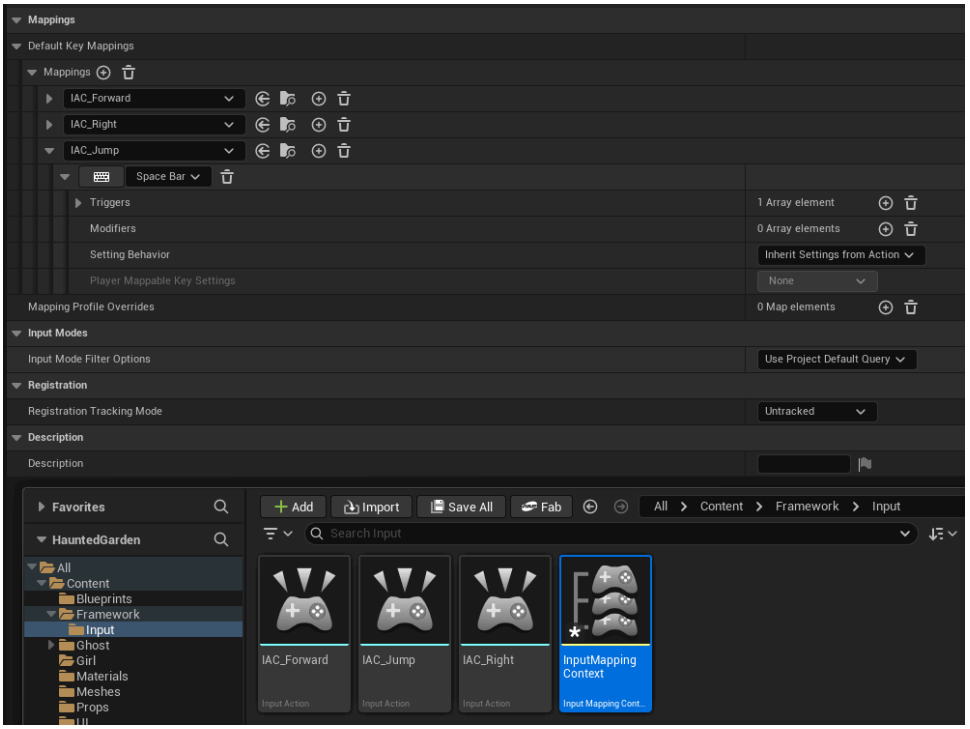
The inclusion of obstacles in the game poses a minor hindrance to the player's movement on the map itself, so one possible solution could be to add a jumping mechanic for the player. The first thing to do would be to create the animations so the character can jump. The jump will be divided into 3 animations:

- **Jump:** Contains the movement of the character when he starts the jump. (**Jump.gif**) [3]

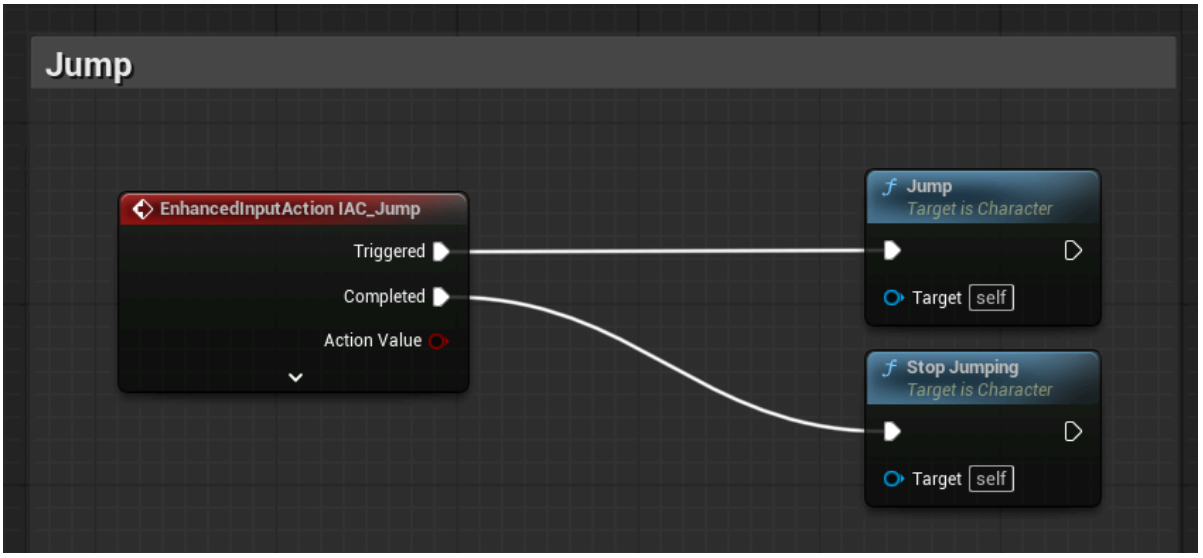
- **Fall:** Follows the Jump animation by moving the character while in the air. (**Fall.gif**) [4]

- **Land:** Once close to the ground, the character regains its original posture to finish the jump.
  (**Land.gif**) [5]

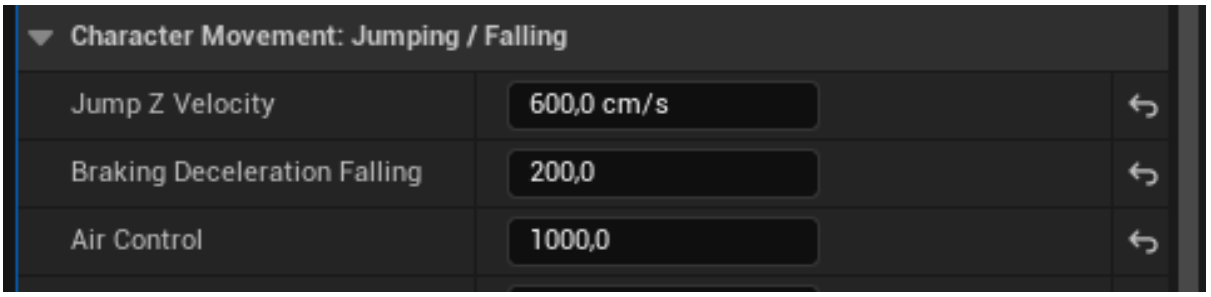
These animations were successfully imported to Unreal Engine and the original Skeletal Mesh of the Player was assigned to all three of them. Now to start building the actual mechanics of jumping, we need to add a new input action for the jump action. To do that, we created **“IAC_Jump”** and modified the **“InputMappingContext”** to include the trigger of this action with the space bar.



The Blueprint of the main character was also modified to include a response to this input, by activating the Jump function, and finishing by calling the Stop Jumping function when the input is completed.



By itself, this would work, but we must adjust some jump parameters in order to balance the jump mechanic. The primary idea is that the player should be able to jump minor obstacles (rock piles and pumpkins) but it shouldn't be able to get past major obstacles (tombstones), so we modified the **Jump Z Velocity**, the **Braking Deceleration Falling** and the **Air Control**, to achieve a more natural and effortless jump.

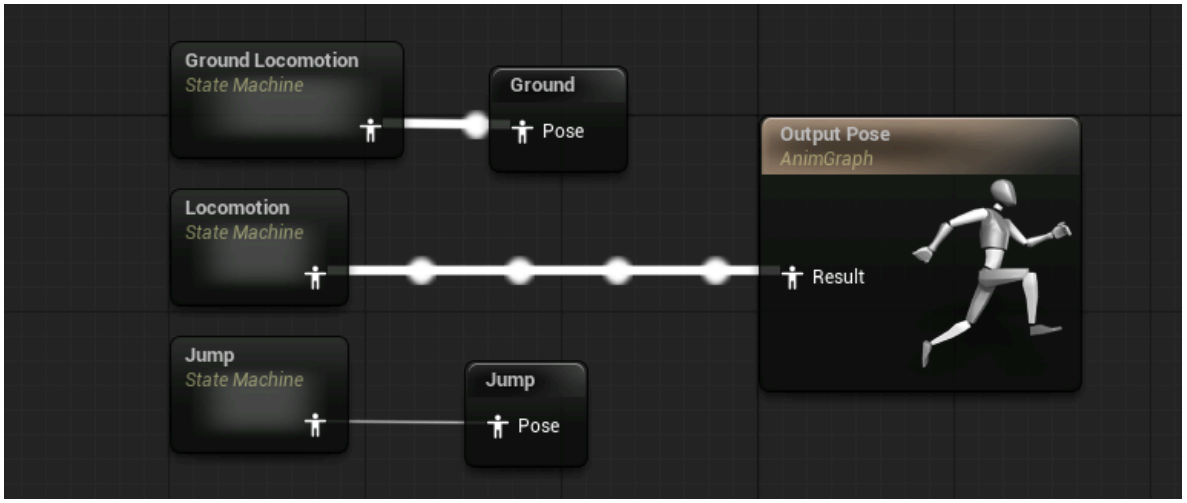


Now the character is able to jump as expected, but there exists a new problem: When trying to jump a minor obstacle, the player could stay still on the top of the obstacles, causing ghosts couldn’t chase him because they can’t reach him. To solve this problem, we will modify the collision of these obstacles so the player can't stay on top of them.

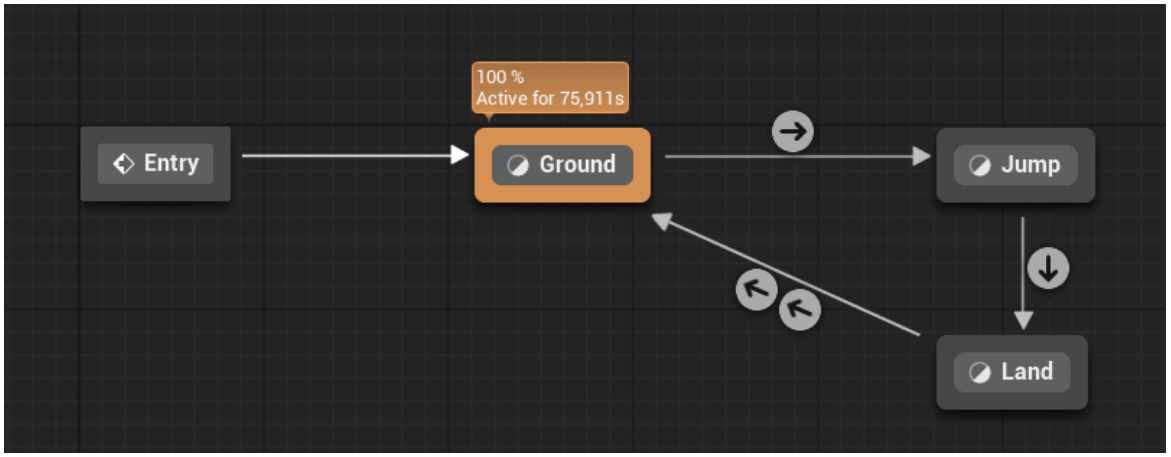
To fix this problem, for both rock piles and pumpkins, we modified the simple collision with the **Auto Convex Collision**, defining a couple of parameters to make a custom collider less precise than the original simple collision. Also, on

both of them we turned on the **Walkable Slope Override** option, which makes the character slope down of the object by a defined angle (10°). With this correctly configured, now the player can perfectly jump over these obstacles without being capable of staying on top of them as seen in **GardenJump.gif** [6] and **ObstaclesJump.gif** [7].

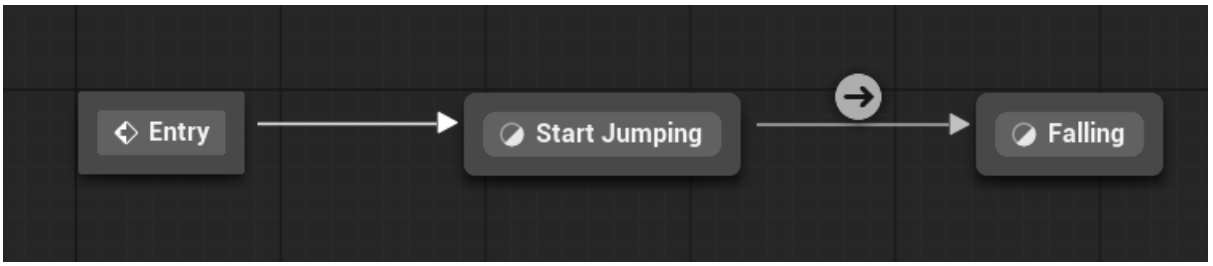
Now the only thing that's left to finish this jump mechanic is to include the animations that we created before on the character blueprint, for that, we'll need to modify the already existing **"ABP_Girl"** to include the animations on the **AnimGraph**. For this to be possible, we modified the EventGraph to include the **"bIsFalling"** variable, that defines when the player is on the air, having that we'll use it as most of the conditions for the multiple transitions in the AnimGraph.



This Animation Graph has three different state machines: One for the Ground Locomotion (Walking and Idling), one for the Jump mechanic and one to mix the previous ones. The Locomotion state machine has a state for the ground locomotion itself, one for the jump and another one for the land animation.



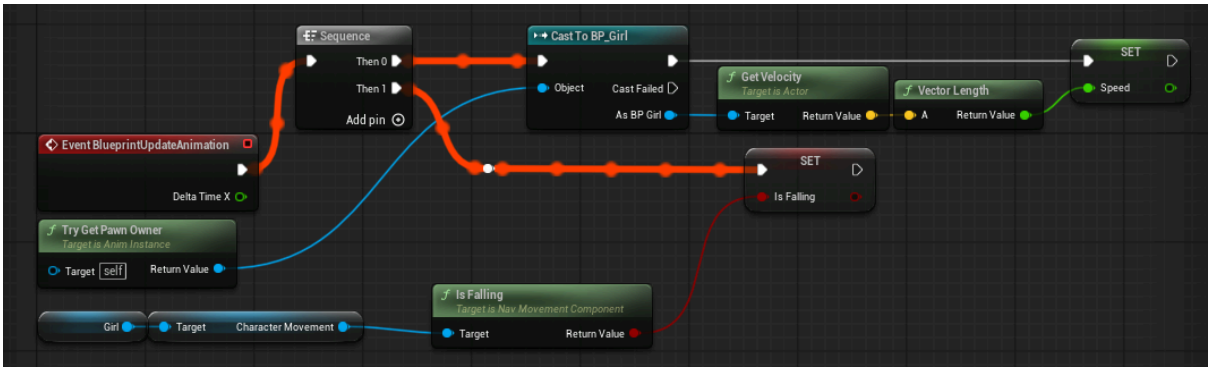
The Ground state takes the cache of the Ground state machine that was previously created to make the character able to walk across the map, but the Jump state takes the cache of the Jump state machine, that's newly created for this mechanic. The condition for advancing to the Jump state is that the character must be falling (on the air), and the Jump state contains the next state machine:



This machine only has two states: One for the start jumping animation and another for the falling animation, the condition to pass from Start Jumping to Falling is that the Jump animation must be completed before, then, automatically passes to the Falling state that reproduces in loop the Fall animation.

To exit the Jump state, the character must not be falling, that is when he is on the ground that the bIsFalling variable turns to false, when that happens, we advance to the final state that is Land. In this state it reproduces the Land animation, and then automatically passes to the Ground state. This state in particular has another transition condition, that is that the bIsFalling variable turns on, while doing the Land animation, the reason of this is that the player could press the spacebar while the animation is playing, so that tells the AnimGraph that it should instantly change before to the Ground locomotion to pass to the Jump states, avoiding the animation to bug unexpectedly.

The bIsFalling variable changes each frame of the Blueprint Animation by updating itself using the **IsFalling** function of the CharacterMovement.



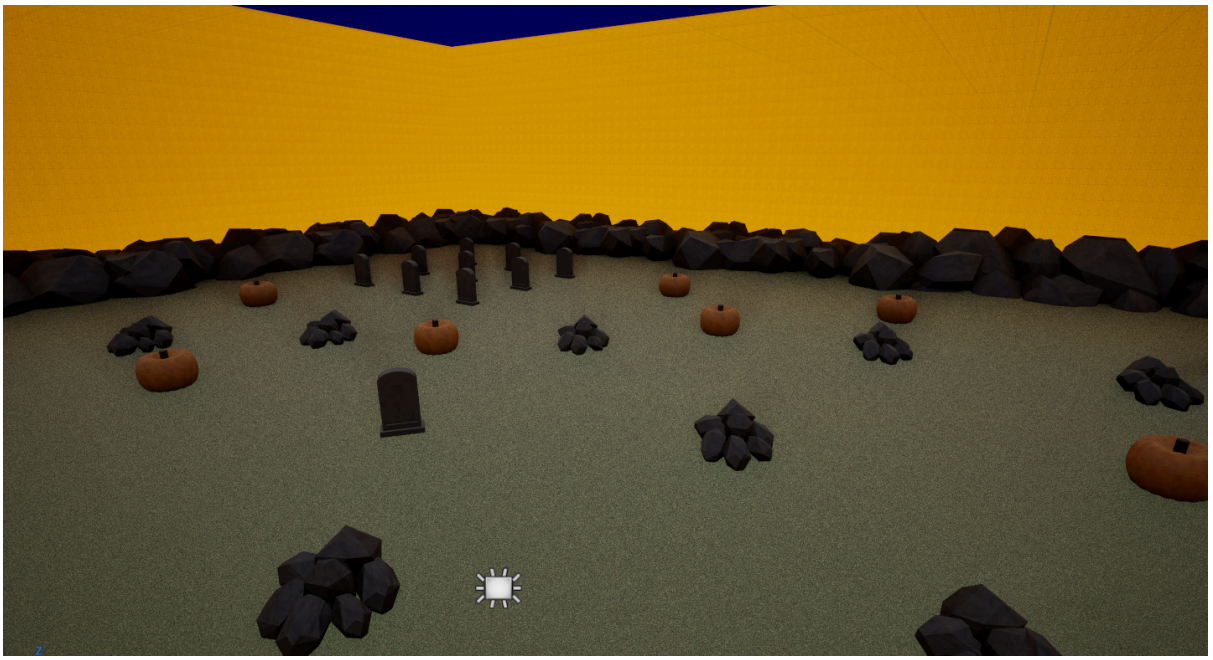
Having all this together, our player can now jump perfectly fine with an animation that makes it more believable (**JumpMechanic.gif**) [8]. Now, apart from the obstacles already existing on the garden, we'll need another mechanic to encourage the player to jump, that will be explained at Section 5.

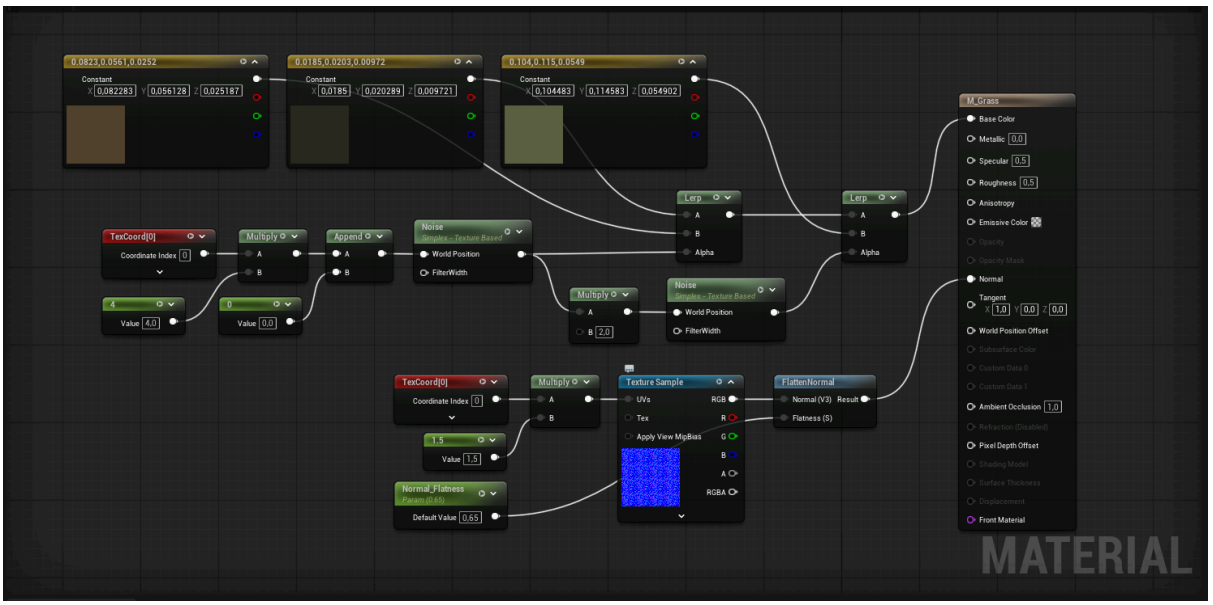
4. Change of Materials (2 points)

In this section we'll upgrade the set of materials present in the base prototype of Haunted Garden, these would be: The ground, the ceiling and the walls. To make it more realistic, the materials would be modified to resemble a creepy ambience, to match a more "haunted" appearance that we want the game to have inside the Unreal Engine 5 material editor.

4.1. Ground

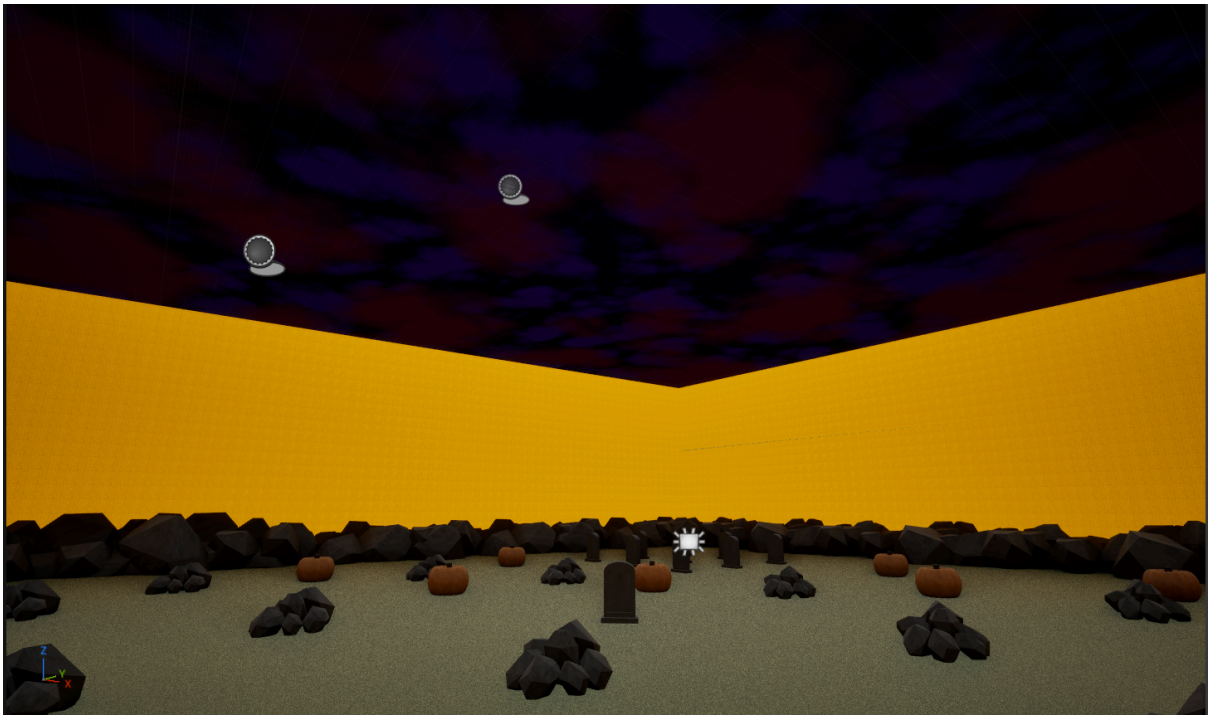
For the ground material, we're going for a dirty, unkempt lawn typical of a haunted garden, so the material features three main colors: a muted grass green, a dirty earth brown, and a mossy gray. After applying a couple of layers of noise and a Normal map to give it more roughness, we achieved the desired look.

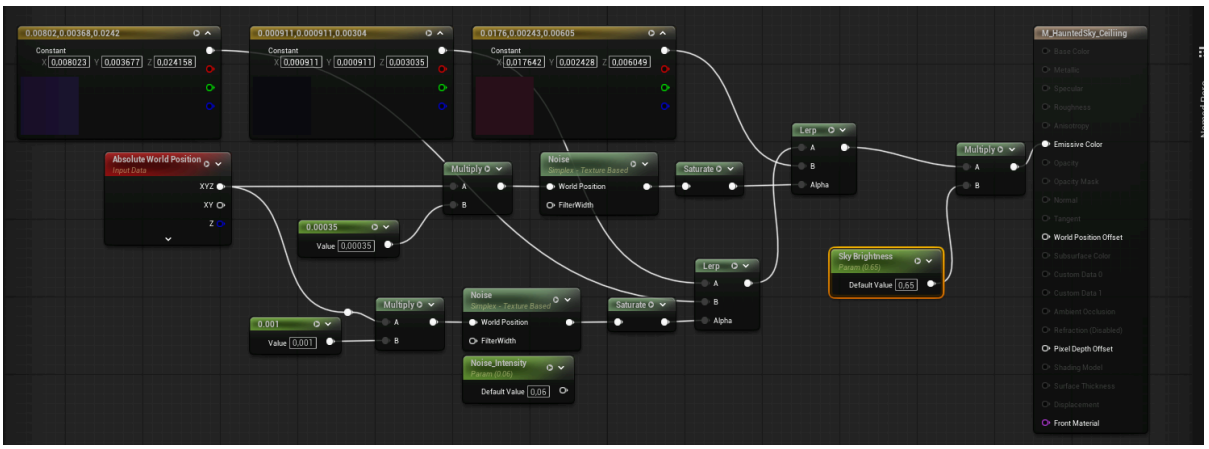




4.2. Ceiling

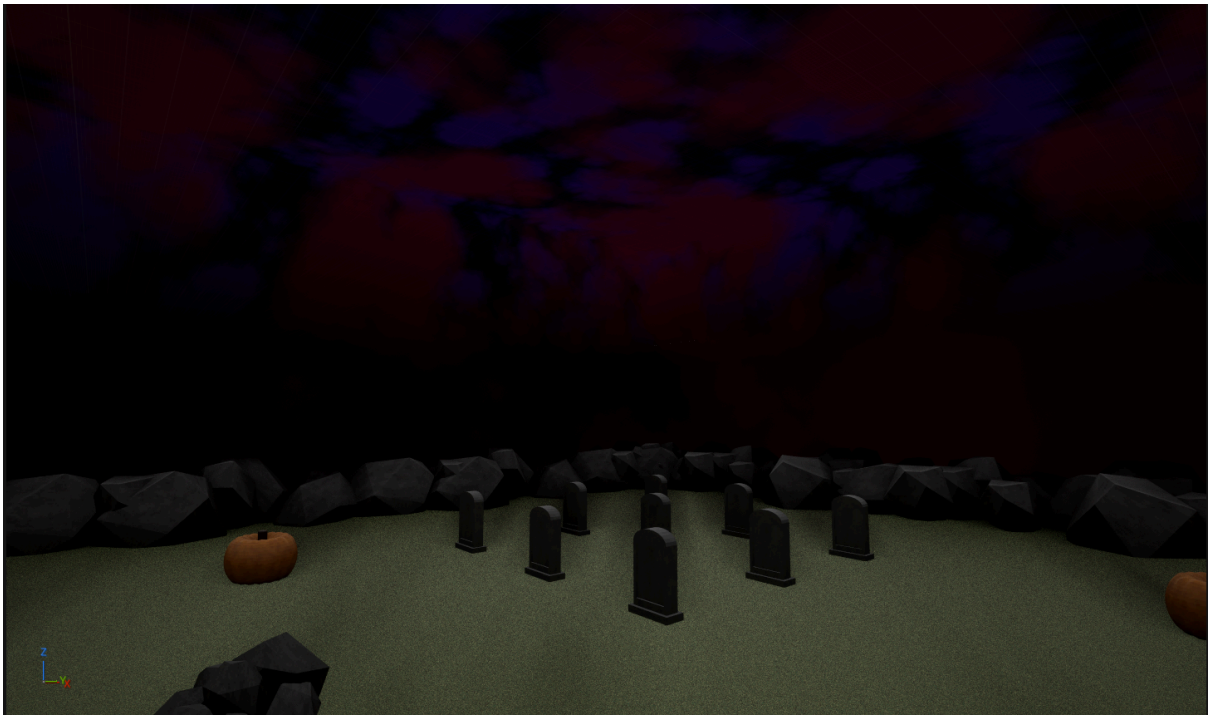
For the ceiling, we're going for a dark and terrifying night sky filled with stormy clouds, so the material features three main colors: an almost black blue, a deep purple, and a muted blood red. By combining two layers of noise at different scales, we created irregular cloud formations that give the sky a more natural and unsettling appearance. We also adjusted the brightness to keep the atmosphere dark and mysterious.

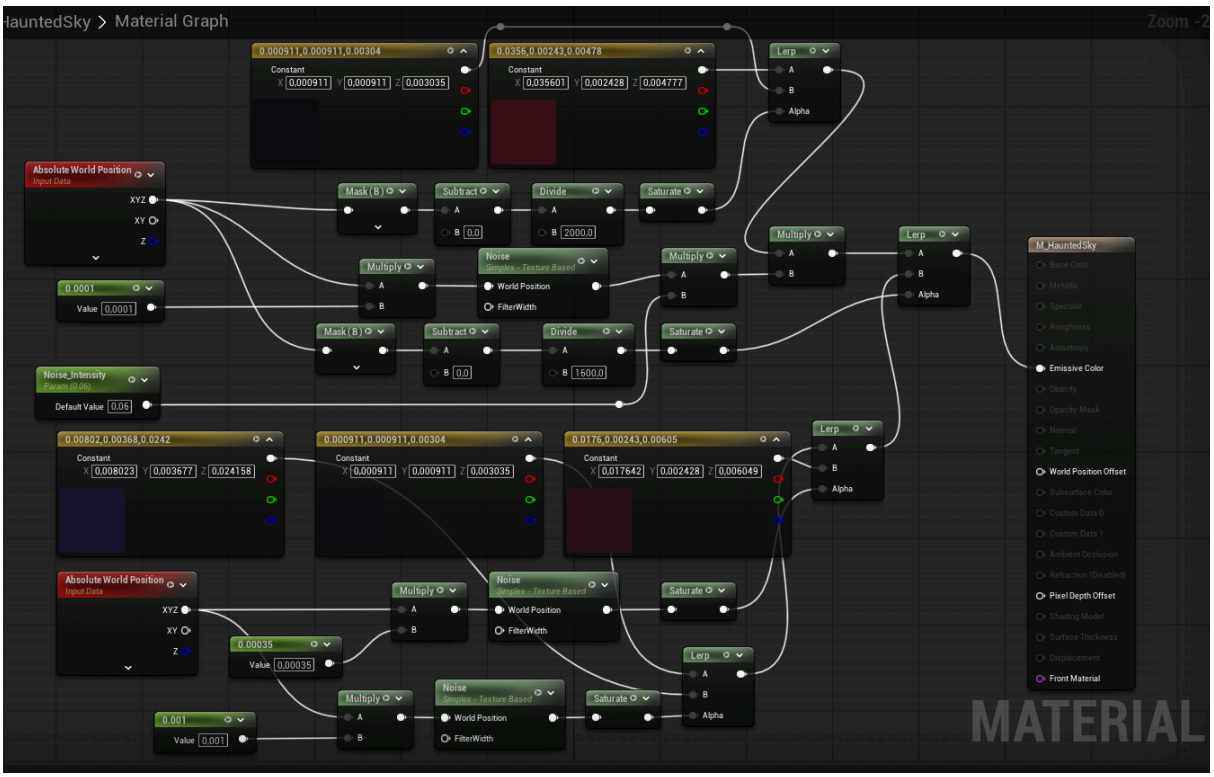




4.3. Walls

For the walls, we wanted to extend the night sky we had already created for the ceiling, making both materials feel like one continuous environment. We started with a vertical gradient transitioning from a deep blood red at the bottom to an almost black tone at the top. Then, we reused the cloud patterns from the ceiling material and added a height-based gradient to gradually blend them into the walls. This way, the clouds fade naturally into the darkness, creating the illusion of a seamless, ominous sky surrounding the haunted garden.



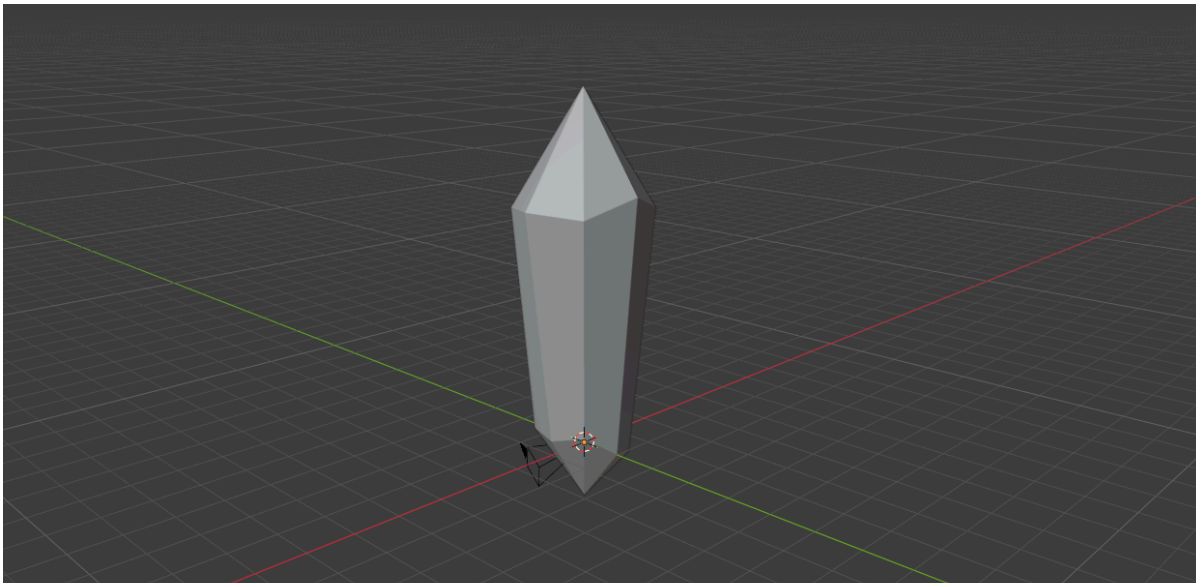


Finally, this would be the look of the game with all three new materials applied:

5. Treasure Collectables (3 points)

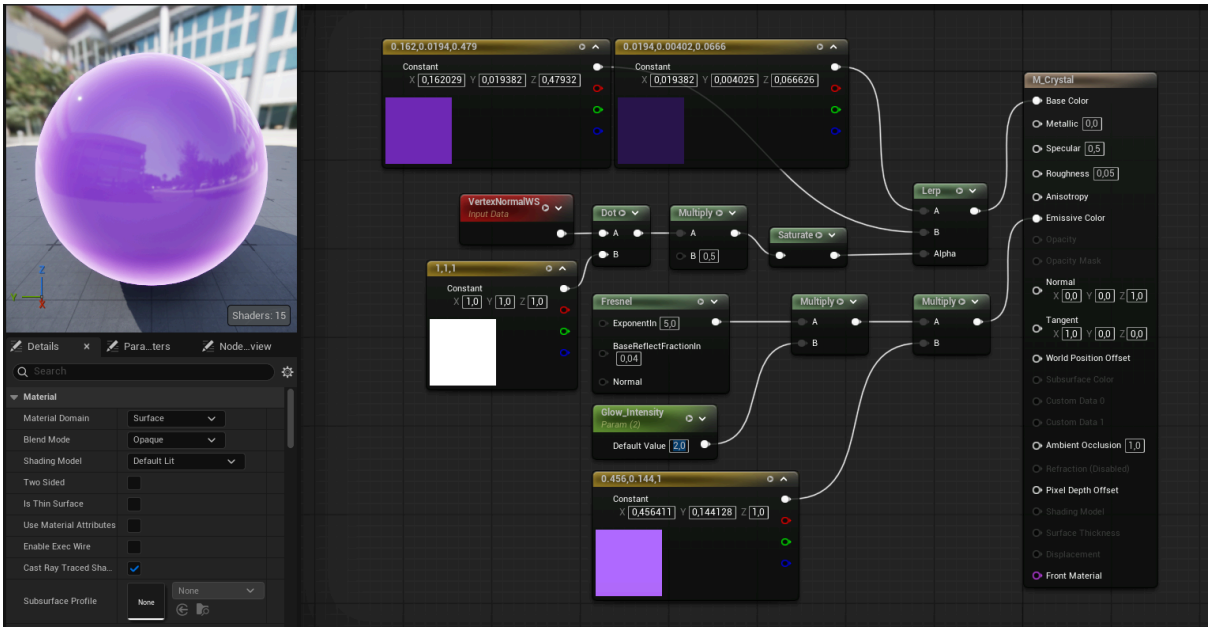
In this final section, we'll introduce the ability to collect items on the map to win the game. To do this, we'll need to: modify the player's Blueprint, modify the Game Mode Blueprint, add the collectibles remaining to complete the level to the UI widget, add the Blueprint for the actor that will represent the collectible, and generate the model for that collectible.

The object we'll be working with is a crystal, which fits in quite well with the aesthetic we've achieved in our game. This crystal was modeled in Blender by extruding a circle with 8 vertices, increasing the scale, combining vertices to create the points, and moving those vertices to create irregularities.

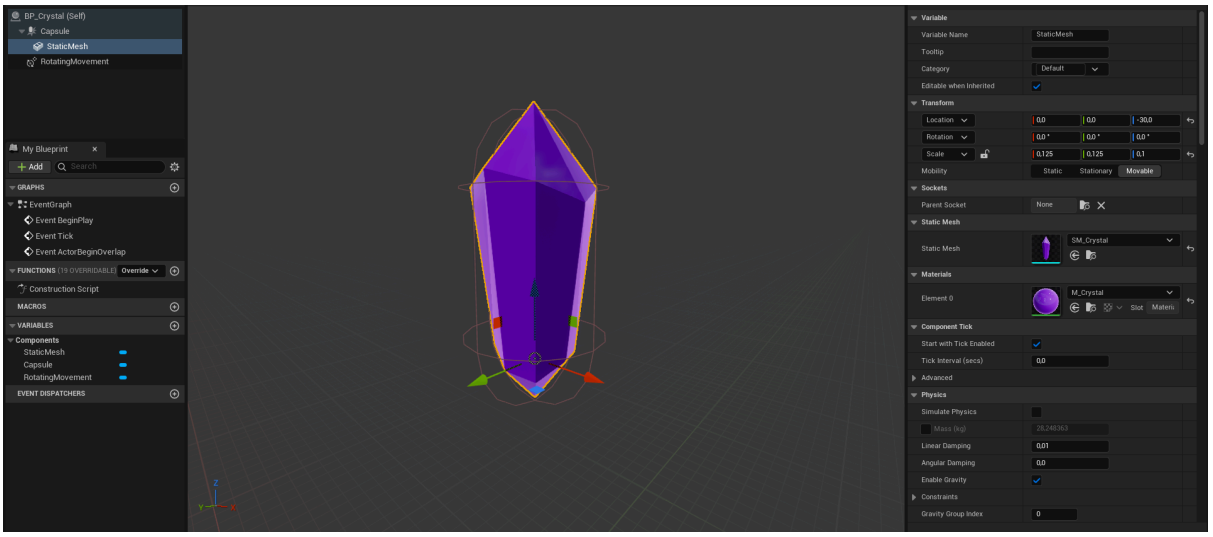


To streamline the process, we define the material directly in the Unreal Engine editor, so we import the FBX file for our crystal model and aim to create a material that reflects bright violet colors.

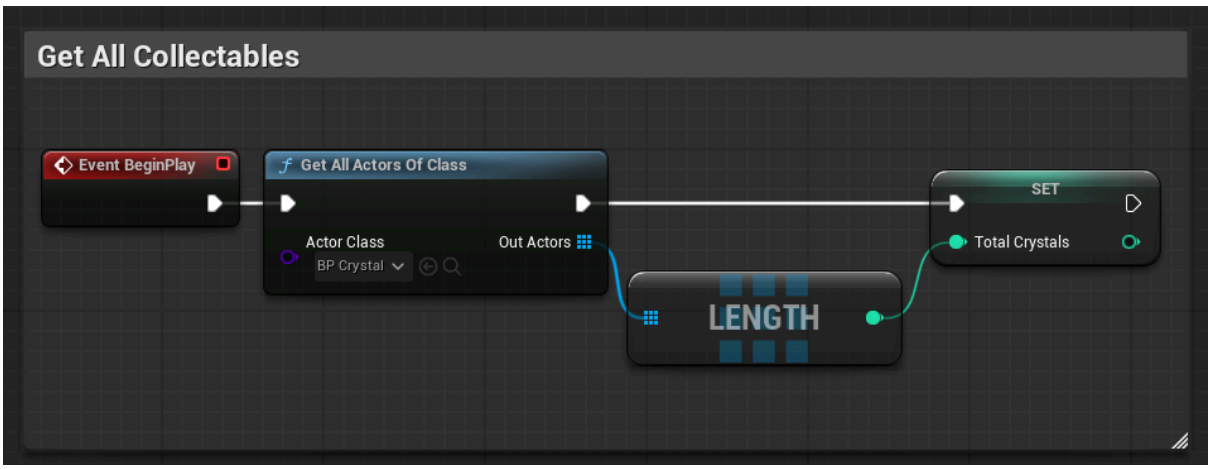
The material features three main colors: a dark purple for the base, a more intense violet to highlight the different faces of the crystal, and a brighter purple for its magical glow. By using the normals of the mesh, we create color variations across its low-poly surfaces, giving the crystal a more defined appearance. We also add a Fresnel effect connected to the Emissive Color to make the edges glow, creating a magical look that helps the collectible stand out in the dark environment of the haunted garden.



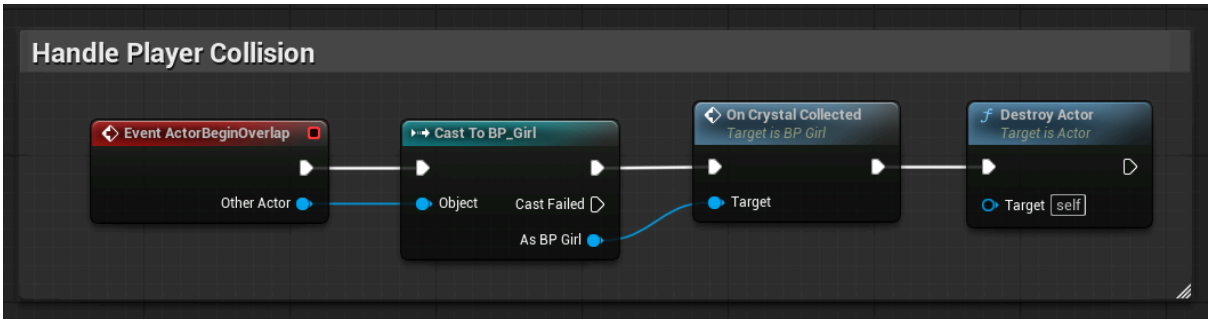
Once we had the crystal model with its material assigned, we proceeded to create the "**BP_Crystal"** blueprint, which will contain the static mesh, the cylinder collider, and a rotation component that we will use later.

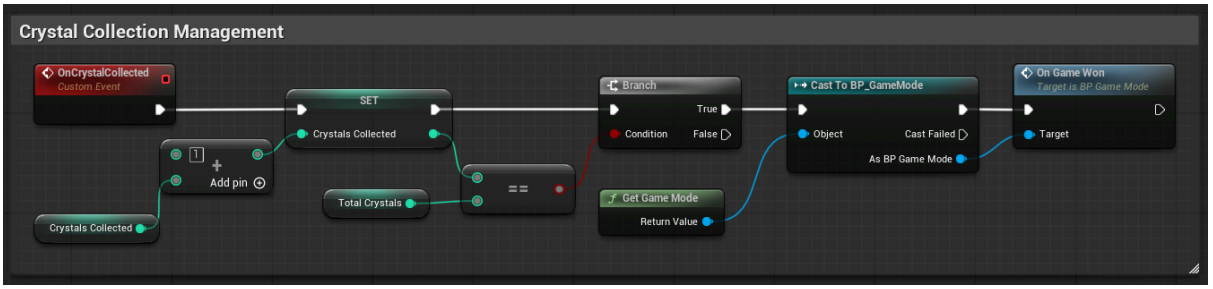


The collection mechanic will work as follows: The player knows the total number of crystals on the map and the number of crystals they have collected, via the variables **"TotalCrystals"** and **"CrystalsCollected"**, respectively.

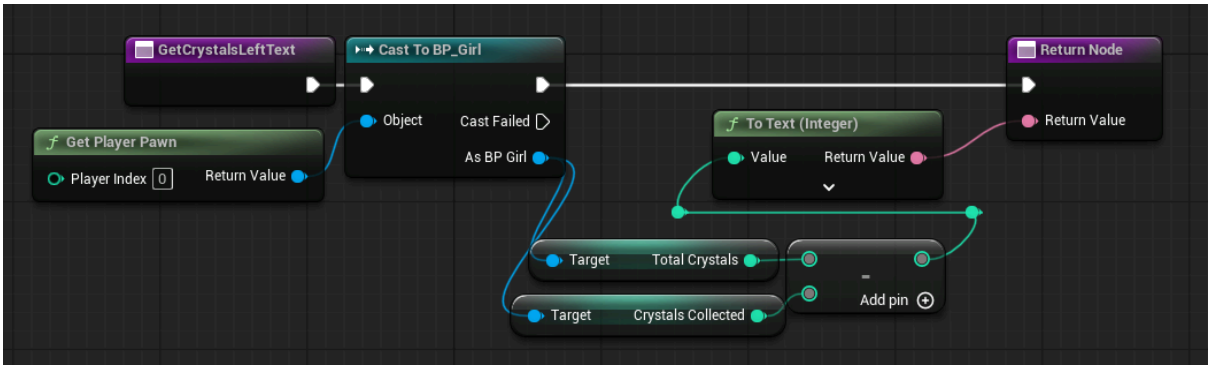


When the player collides with the crystal actor, the crystal blueprint will send an **“OnCrystalCollected”** event signal to the player and self-destruct. The player will increment the number of crystals collected by one and verify that the number of crystals collected equals the total number of crystals; if so, it will call the “**OnGameWon”** event of the “**BP_GameMode”**, which will transition to the corresponding level displaying the victory screen.



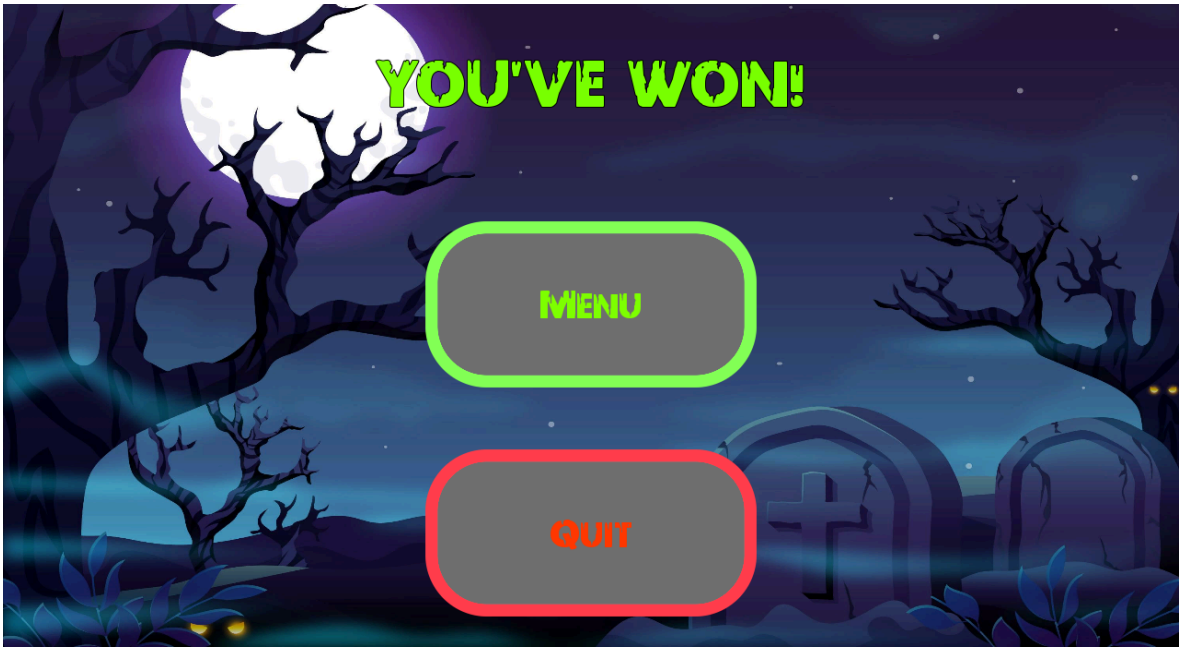


The UI was modified to include the “Crystals Left” text, which displays the number of crystals remaining from the **“GetCrystalsLeftText”** function.

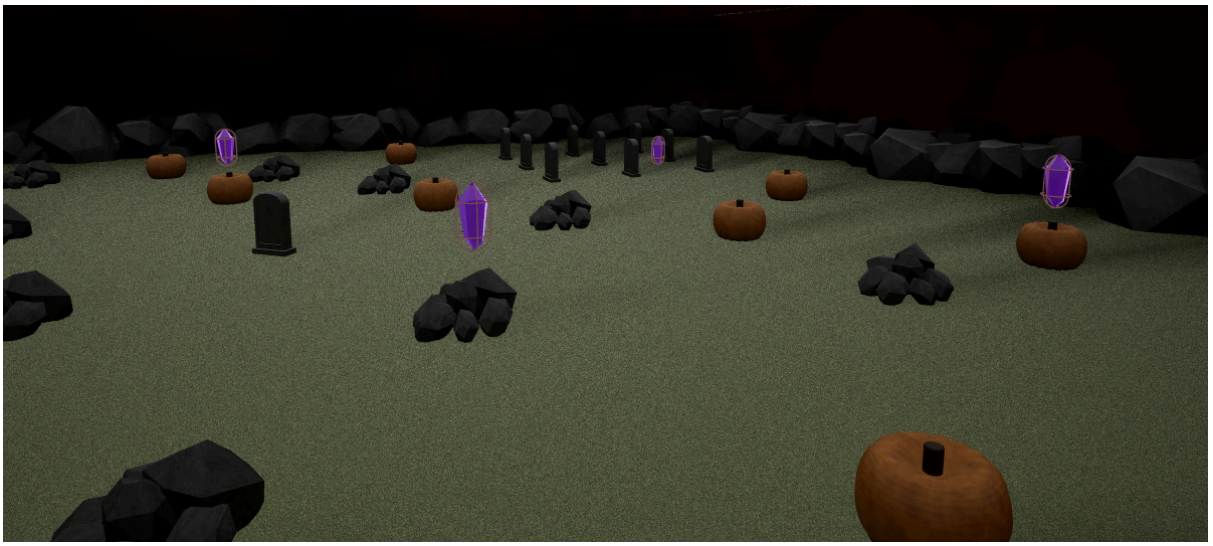


In addition, the rotation component we included in the crystal's blueprint is designed to maintain rotation at a constant speed without relying on a more complex animation.

Finally, a completely new level was added to correspond to the victory menu level called **“WinMenu”**, for which a separate game mode and HUD blueprint had to be created. For this menu, we simply used the same background as the game's original menu, placing a victory message at the top and two buttons at the bottom: one to return to the main menu and another to exit the game.



With all of this properly configured, about 10 crystals were added and scattered throughout the map; if the player is able to collect them all, they will win the game. Additionally, to encourage the player to use all the mechanics at their disposal, collecting some of these crystals will require jumping over the predefined obstacles, forcing the player to master their jumping skills.



6. Final Result

In summary, the upgrades implemented have transformed Haunted Garden from a simple prototype into a much more dynamic and visually cohesive gameplay experience.

The addition of 3D obstacles adds structural complexity to the map, while the new jumping mechanic and character animation state machine provide the player with essential mobility options to navigate these hazards. Furthermore, custom materials for the ground, ceiling, and surrounding walls elevate the environment's haunted atmosphere. Finally, the inclusion of collectible crystals, integrated UI tracking, and a dedicated victory screen complete the primary game loop, offering clear objectives and rewarding player skill. All of the Blender files used across the project are available at the project GitHub repository [9]. This video [10] features a complete gameplay recording of the new Haunted Garden, showcasing all the mechanics and features mentioned throughout the report.

7. Bibliography

1. Stone Material in Blender: youtu.be/WQ2xF40s1NM
2. Pumpkin Modelling in Blender: youtu.be/N9afBWf7FVY
3. Jump Gif: github.com/alu0101549491/MSDV-HauntedGarden/blob/main/Media/Jump.gif
4. Fall Gif: github.com/alu0101549491/MSDV-HauntedGarden/blob/main/Media/Fall.gif
5. Land Gif: github.com/alu0101549491/MSDV-HauntedGarden/blob/main/Media/Land.gif
6. Garden Jump Gif: github.com/alu0101549491/MSDV-HauntedGarden/blob/main/Media/GardenJump.gif
7. Obstacles Jump Gif: github.com/alu0101549491/MSDV-HauntedGarden/blob/main/Media/ObstaclesJump.gif
8. Jump Mechanic Gif: github.com/alu0101549491/MSDV-HauntedGarden/blob/main/Media/JumpMechanic.gif
9. Blender files, FBX files and Texture files: github.com/alu0101549491/MSDV-HauntedGarden/blob/main/Blends
10. Haunted Garden Gameplay: youtu.be/UrvJTxg0pf8